

Maxwell's Equations on FPGA

FDTD Implementation — Complete Technical Reference

EE2 Design Project — Imperial College London

Document Scope

This document covers Maxwell's equations from first principles, the FDTD numerical method in full mathematical detail, how to implement FDTD in Verilog on the Pynq-Z1 FPGA, how to connect the physical conducting paper measurement as boundary conditions, the full 3D visualisation pipeline, and the Poynting vector computation. Every section includes implementation detail sufficient to write the actual Verilog and Python code.

PART 1 — THE PHYSICS

1. Maxwell's Equations — From First Principles

1.1 The Four Equations

Maxwell's equations are four coupled partial differential equations that together describe all classical electromagnetic phenomena. Every electrical engineering concept — capacitors, inductors, antennas, waveguides, optical fibres, wireless communication — follows from these four equations.

Equation	Differential Form	Physical Meaning
Gauss's Law (Electric)	$\nabla \cdot \mathbf{E} = \rho / \epsilon_0$	Electric field lines originate on positive charges and end on negative charges
Gauss's Law (Magnetic)	$\nabla \cdot \mathbf{B} = 0$	No magnetic monopoles exist — field lines always form closed loops
Faraday's Law	$\nabla \times \mathbf{E} = -\partial \mathbf{B} / \partial t$	A time-varying magnetic field creates a circulating electric field
Ampere-Maxwell Law	$\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \epsilon_0 \partial \mathbf{E} / \partial t$	Currents and time-varying electric fields create circulating magnetic fields

1.2 Why These Four Equations Contain Everything

1.2.1 Gauss's Law for E — $\nabla \cdot \mathbf{E} = \rho / \epsilon_0$

The divergence of $\mathbf{E}$ at any point equals the charge density ρ divided by the permittivity ϵ_0 . Divergence measures how much a vector field 'spreads out' from a point. Where $\rho = 0$ (no free charges, as in your conducting paper between the electrodes), this becomes $\nabla \cdot \mathbf{E} = 0$. Combined with the fact that $\mathbf{E} = -\nabla V$ in static cases, this directly gives Laplace's equation $\nabla^2 V = 0$. So Laplace is just a special case of Maxwell — specifically the static, charge-free case.

1.2.2 Gauss's Law for B — $\nabla \cdot \mathbf{B} = 0$

Magnetic field lines never start or end anywhere — they always form closed loops. There are no magnetic monopoles (isolated north or south poles). This is a constraint on $\mathbf{B}$ rather than a source equation — it tells you $\mathbf{B}$ must always be divergence-free.

1.2.3 Faraday's Law — $\nabla \times \mathbf{E} = -\partial \mathbf{B} / \partial t$

This is where electricity and magnetism first become coupled. If $\mathbf{B}$ changes in time at some point in space, a circulating $\mathbf{E}$ field is created around that point. This is the physical principle behind

transformers, generators, and wireless power transfer. The curl operator ($\nabla \times$) measures the rotation or circulation of a vector field around a point.

In your project: when you drive the electrodes with an AC signal, B is oscillating in time. Faraday's law says this creates a small circulating E field. At low frequencies (1kHz) this effect is tiny — but at higher effective frequencies in your FDTD simulation it becomes the dominant physics.

1.2.4 Ampere-Maxwell Law — $\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \epsilon_0 \partial \mathbf{E} / \partial t$

Two things create a circulating B field: real current density J (moving charges) and displacement current $\mu_0 \epsilon_0 \partial \mathbf{E} / \partial t$ (time-varying E field). The displacement current term was Maxwell's crucial addition to Ampere's original law. Without it, the equations are inconsistent. With it, they predict electromagnetic waves.

1.3 How Maxwell's Equations Predict EM Waves

Take the curl of Faraday's law:

$$\nabla \times (\nabla \times \mathbf{E}) = -\partial (\nabla \times \mathbf{B}) / \partial t$$

Use the vector identity $\nabla \times (\nabla \times \mathbf{E}) = \nabla(\nabla \cdot \mathbf{E}) - \nabla^2 \mathbf{E}$, and in free space $\nabla \cdot \mathbf{E} = 0$:

$$-\nabla^2 \mathbf{E} = -\partial (\nabla \times \mathbf{B}) / \partial t$$

Substitute the Ampere-Maxwell law for $\nabla \times \mathbf{B}$ (with $\mathbf{J} = 0$ in free space):

$$-\nabla^2 \mathbf{E} = -\mu_0 \epsilon_0 \partial^2 \mathbf{E} / \partial t^2$$

Rearranging:

$$\nabla^2 \mathbf{E} = \mu_0 \epsilon_0 \partial^2 \mathbf{E} / \partial t^2$$

This is the wave equation. Its solution is a wave propagating at speed:

$$c = 1/\sqrt{\mu_0 \epsilon_0} = 3 \times 10^8 \text{ m/s}$$

Maxwell recognised this as the speed of light and immediately concluded that light is an electromagnetic wave. This is the central unifying discovery of 19th-century physics. Your FDTD solver numerically integrates exactly this equation — and will show you electromagnetic waves propagating across the screen in real time.

1.4 Why Laplace is Not Enough

Laplace's equation $\nabla^2 V = 0$ describes only electrostatics — the situation where nothing changes in time. As soon as you have time-varying fields, you need the full Maxwell equations. The new physics that appears includes:

- Electromagnetic wave propagation — energy travelling through space at the speed of light
- Radiation — an accelerating charge radiates energy outward permanently
- Induction — time-varying B fields drive currents in nearby conductors
- Displacement current — time-varying E fields act like real currents in terms of generating B

- Skin effect — high-frequency currents concentrate at conductor surfaces
- Resonance — EM energy can be stored and exchanged between E and B fields

All of these phenomena are invisible to Laplace but visible to FDTD. Your FPGA will simulate all of them simultaneously.

PART 2 — THE FDTD METHOD

2. FDTD — Finite Difference Time Domain

2.1 The Core Idea

FDTD is a method for numerically solving Maxwell's equations by discretising both space and time into a grid, then marching the solution forward one time step at a time. It was introduced by Kane Yee in 1966 and remains the most widely used method for computational electromagnetics.

The key insight is that Maxwell's equations in the time domain tell you how the fields change from one moment to the next. If you know E and B everywhere right now, the curl equations tell you exactly how fast they are changing — so you can compute E and B a moment later. Repeat indefinitely to march the solution forward in time.

2.2 The 2D TE Mode — Your Simulation

Implementing full 3D FDTD would require storing six field components (E_x , E_y , E_z , B_x , B_y , B_z) at every point in a 3D volume — far too much memory and compute for your FPGA. Instead you use 2D FDTD in the Transverse Electric (TE) mode.

TE mode assumes the simulation is a 2D cross-section with no variation in the z -direction ($\partial/\partial z = 0$). This gives exactly three non-zero field components:

- $E_x(x,y,t)$ — electric field in x -direction
- $E_y(x,y,t)$ — electric field in y -direction
- $B_z(x,y,t)$ — magnetic field in z -direction (perpendicular to the simulation plane)

The other three components (E_z , B_x , B_y) are identically zero in TE mode. Maxwell's four equations reduce to three update equations for these three components — exactly what FDTD needs.

2.3 Deriving the FDTD Update Equations

2.3.1 Starting from Maxwell's Curl Equations

In 2D TE mode with $\partial/\partial z = 0$, the Faraday and Ampere-Maxwell equations expand to:

$$\frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x} \quad (\text{from Faraday: } \nabla \times E = -\frac{\partial B}{\partial t}, \text{ } z\text{-component})$$

$$\frac{\partial E_x}{\partial t} = (1/\mu_0\epsilon_0) \frac{\partial B_z}{\partial y} \quad (\text{from Ampere-Maxwell: } \nabla \times B = \mu_0\epsilon_0\frac{\partial E}{\partial t}, \text{ } x\text{-component})$$

$$\frac{\partial E_y}{\partial t} = -(1/\mu_0\epsilon_0) \frac{\partial B_z}{\partial x} \quad (\text{from Ampere-Maxwell: } \nabla \times B = \mu_0\epsilon_0\frac{\partial E}{\partial t}, \text{ } y\text{-component})$$

These three equations are exact. FDTD approximates the spatial and temporal derivatives using finite differences.

2.3.2 Finite Difference Approximation

A first-order derivative is approximated by a central difference:

$$\partial f / \partial x \approx (f(x + \Delta x / 2) - f(x - \Delta x / 2)) / \Delta x$$

This is second-order accurate — the error is proportional to $(\Delta x)^2$. The key point is that the derivative at position x is evaluated using values at $x \pm \Delta x / 2$ — half a grid cell away. This naturally leads to the Yee grid where E and B are stored at offset positions.

2.3.3 The Yee Grid

Kane Yee's genius was to store E_x , E_y , and B_z at different positions within each cell, offset by half a grid spacing. This is not an approximation — it is the natural grid for the central difference scheme and gives second-order accuracy with no additional cost.

Component	Grid Position	Why Here
$E_x(i,j)$	$(i+1/2)\Delta x, j\Delta y$	Halfway between B_z nodes in x , at integer y
$E_y(i,j)$	$i\Delta x, (j+1/2)\Delta y$	At integer x , halfway between B_z nodes in y
$B_z(i,j)$	$(i+1/2)\Delta x, (j+1/2)\Delta y$	At the centre of the cell, surrounded by E nodes

In Verilog this means three separate 2D arrays (block RAMs), each sized $NX \times NY$, where the index (i,j) implicitly encodes the offset position. There is no explicit coordinate storage — the position is determined by the array index.

2.3.4 The Leapfrog Time Scheme

E and B are also offset in time by half a time step. B is updated at integer time steps, E is updated at half-integer time steps. They leapfrog past each other:

$$\begin{aligned}
 t = 0: & \quad E^0 \text{ known (initial conditions)} \\
 t = \Delta t / 2: & \quad B^{(1/2)} \text{ computed from } E^0 \\
 t = \Delta t: & \quad E^1 \text{ computed from } B^{(1/2)} \\
 t = 3\Delta t / 2: & \quad B^{(3/2)} \text{ computed from } E^1 \\
 & \quad \dots \text{ and so on}
 \end{aligned}$$

This leapfrog scheme is time-reversible and conserves energy — the simulation does not gain or lose energy artificially over long run times. This is essential for physically correct wave propagation.

2.4 The Final FDTD Update Equations

Applying the central differences in space and the leapfrog scheme in time to the three curl equations gives the FDTD update equations. These are what your Verilog implements:

Update 1 — Bz at half time step

$$\begin{aligned} Bz^{(n+\frac{1}{2})}(i,j) &= Bz^{(n-\frac{1}{2})}(i,j) \\ &+ (\Delta t / \Delta x) [Ex^n(i,j+1) - Ex^n(i,j)] \\ &- (\Delta t / \Delta y) [Ey^n(i+1,j) - Ey^n(i,j)] \end{aligned}$$

Update 2 — Ex at full time step

$$\begin{aligned} Ex^{(n+1)}(i,j) &= Ex^n(i,j) \\ &+ (\Delta t / \mu_0 \epsilon_0 \Delta y) [Bz^{(n+\frac{1}{2})}(i,j) - Bz^{(n+\frac{1}{2})}(i,j-1)] \end{aligned}$$

Update 3 — Ey at full time step

$$\begin{aligned} Ey^{(n+1)}(i,j) &= Ey^n(i,j) \\ &- (\Delta t / \mu_0 \epsilon_0 \Delta x) [Bz^{(n+\frac{1}{2})}(i,j) - Bz^{(n+\frac{1}{2})}(i-1,j)] \end{aligned}$$

What These Equations Say Physically

Each field component at every grid point is updated using only its immediate neighbours (one cell away) and the value at the previous time step. No global information is needed — the update is completely local. This is why FDTD is perfectly suited to parallel hardware: every cell can update simultaneously, independently of all other cells. This is the embarrassingly parallel structure the project requires.

2.5 The CFL Stability Condition

FDTD is only stable if the time step Δt is small enough relative to the spatial step Δx . Physically, this means the simulated wave must not travel more than one grid cell per time step. The Courant-Friedrichs-Lewy (CFL) condition in 2D is:

$$\Delta t \leq \Delta x / (c_{eff} \times \sqrt{2})$$

Where c_{eff} is the effective wave speed in your simulation. If you violate this condition, the simulation becomes numerically unstable — field values grow exponentially and quickly overflow to infinity. In Verilog, this manifests as fixed-point overflow and the display fills with garbage.

To guarantee stability, choose:

$$\Delta t = 0.99 \times \Delta x / (c_{eff} \times \sqrt{2}) \quad (1\% \text{ safety margin})$$

2.6 The Speed Scaling Problem and Solution

2.6.1 The Problem

In free space, $c = 3 \times 10^8$ m/s. To see a full wavelength of a 1kHz wave, you would need a simulation domain 300km wide. This is completely impractical. Even at 1GHz, the wavelength is 30cm — possible but requires a very fine grid to resolve well.

2.6.2 The Solution — Effective Permittivity

In a material with relative permittivity ϵ_r , the wave speed is reduced:

$$c_{\text{eff}} = c / \sqrt{\epsilon_r}$$

By choosing a very large ϵ_r in the simulation, you can reduce c_{eff} arbitrarily. This is mathematically legitimate — you are simulating wave propagation in a high-permittivity medium. The wave behaviour (reflection, interference, diffraction) is identical in form to free space. Only the speed changes.

2.6.3 Choosing Your Simulation Parameters

You want to see interesting wave phenomena on a display-sized domain. Here is how to choose the parameters:

Parameter	Symbol	Chosen Value	Reasoning
Grid size	$N_X \times N_Y$	256×256	Good resolution, fits in Pynq block RAM
Cell size	$\Delta x = \Delta y$	1 (normalised)	Simulation units — physical scale set by ϵ_r
Effective speed	c_{eff}	0.5 (normalised)	$c_{\text{eff}} = c / \sqrt{\epsilon_r}$, choose $\epsilon_r = 4$
Time step	Δt	0.35	$0.99 \times 1 / (0.5 \times \sqrt{2}) \approx 0.35 \times \text{CFL}$ satisfied
Update coefficient	C_b	$\Delta t / (\mu_0 \epsilon_0 \Delta x) \rightarrow c_{\text{eff}}^2 \times \Delta t / \Delta x$	Coefficient in the FDTD equations
Source frequency	f_0	0.05 (normalised)	10–20 cells per wavelength — well resolved
Wavelength	λ	$c_{\text{eff}} / f_0 = 10$ cells	Visible interference patterns on 256-cell grid

With these parameters, a 256×256 grid shows approximately 25 wavelengths across the domain — enough to see multiple interference fringes, standing waves, and diffraction patterns simultaneously.

2.6.4 Mapping to Physical Units for Display

When rendering, the normalised simulation coordinates map to pixels directly. The colour represents field amplitude E_x , E_y , or B_z . The fact that the speed is scaled does not affect the visual appearance of interference patterns, standing waves, or diffraction — these are determined by the ratio of wavelength to domain size, which you have chosen to give interesting patterns.

2.7 Source Injection

To drive the simulation you inject a time-varying field value at one or more source points. This represents your physical electrodes driving the EM field.

2.7.1 Hard Source

The simplest approach: force E_x or E_y at one grid point to follow a prescribed waveform, overriding the FDTD update at that point:

$$Ex(isrc, jsrc) = A \times \sin(2\pi \times f_0 \times n \times \Delta t)$$

Where n is the time step counter. This is called a hard source. The electrode drives the field directly with no reflection.

2.7.2 Soft Source (Total-Field/Scattered-Field)

A more sophisticated approach adds the source waveform to the FDTD update rather than replacing it:

$$Ex(isrc, jsrc) += A \times \sin(2\pi \times f_0 \times n \times \Delta t)$$

This allows reflected waves to pass back through the source point without artificial scattering — more physically correct but slightly more complex to implement. Use hard source first and switch to soft source as an extension.

2.7.3 Multiple Sources

Multiple sources at different positions create interference patterns. Each source injects at the same frequency but potentially different phase. A phase difference of π (180°) between two sources gives destructive interference midway between them. This is directly analogous to Young's double-slit experiment for light — one of the fundamental demonstrations of wave behaviour.

In Verilog, multiple sources are handled by a list of $(x, y, \text{amplitude}, \text{phase})$ tuples stored in registers. The source injection logic iterates through this list each time step and adds contributions to the field.

2.8 Absorbing Boundary Conditions — Perfectly Matched Layer

Without special treatment at the domain boundaries, waves reflect off the edges and propagate back into the simulation. This creates standing waves that obscure the physics you want to show. The Perfectly Matched Layer (PML) absorbs outgoing waves with essentially no reflection.

2.8.1 How PML Works

PML adds a thin border region (typically 8–16 cells wide) around the domain where the fields are gradually attenuated. The attenuation profile is designed so that waves entering the PML are absorbed without any reflection at the PML interface — the impedance matches perfectly regardless of angle or frequency.

The attenuation is implemented by adding a conductivity σ to the update equations in the PML region:

$$Ex_update_in_PML = Ex \times (1 - \sigma\Delta t/\epsilon_0) + (\Delta t/\epsilon_0) \times (\partial B_z/\partial y)$$

The conductivity σ increases from zero at the inner PML boundary to a maximum at the outer edge, following a polynomial profile (typically cubic) to minimise reflections at the interface.

2.8.2 Simplified PML for FPGA

Full PML requires splitting the field components (E_x becomes $E_{xy} + E_{xz}$ in 3D) which doubles memory requirements. For 2D TE mode a simpler approach works well: the Convolutional PML (CPML) or simply a graded conductivity profile applied to the standard update equations. In practice for your project, use a simple exponential decay:

$$field_at_boundary = field \times decay_factor^{(distance_from_edge)}$$

Where $decay_factor \approx 0.90$ – 0.95 per cell in the boundary region (8 cells wide). This is not perfect PML but gives acceptable absorption for a teaching demonstration and is trivial to implement — just a multiplication by a stored coefficient at each boundary cell.

3. FPGA Architecture for FDTD

3.1 Memory Architecture

The FDTD simulation requires storing three 2D field arrays. For a 256×256 grid at 16-bit fixed-point resolution:

Array	Size	Bits	Block RAMs (36Kb each)
Ex[256][256]	65,536 elements	16 bits each → 1,048,576 bits	29 BRAMs
Ey[256][256]	65,536 elements	16 bits each → 1,048,576 bits	29 BRAMs
Bz[256][256]	65,536 elements	16 bits each → 1,048,576 bits	29 BRAMs
PML coefficients	512 (edge values)	16 bits each → 8,192 bits	1 BRAM
Total			~88 BRAMs of 220 available on Pynq-Z1 ✓

The Pynq-Z1 (Zynq XC7Z020) has 140×36Kb = 4.9Mb of block RAM — comfortably fits all three arrays with room for the pixel buffer and other logic.

3.1.1 Memory Access Pattern

Each FDTD update at pixel (i,j) requires reading up to 4 neighbouring values: (i,j), (i±1,j), (i,j±1). This creates a memory bandwidth challenge — you need multiple simultaneous reads. The solution is to use dual-port BRAMs: one port for reading the current value, one port for writing the updated value. By processing rows sequentially, neighbouring values can be cached in registers, minimising redundant BRAM accesses.

3.1.2 Ping-Pong Buffering

While the FDTD engine is computing the next time step (writing to buffer B), the renderer reads the current time step (reading from buffer A). After the FDTD step completes, the roles swap. This means the renderer always has a complete, consistent field snapshot to render — never a half-updated frame.

3.2 Fixed-Point Number Representation

3.2.1 Format Choice

All FDTD arithmetic uses Q3.13 fixed-point format: 3 integer bits, 13 fractional bits, 1 sign bit = 17 bits total (use 18-bit to fit Xilinx DSP48E1 input width). This gives:

- Range: -4.0 to +3.9999 — field values normalised to fit this range
- Resolution: $2^{-13} \approx 0.000122$ — sufficient for wave amplitudes
- Multiplication result: 36-bit intermediate, truncated back to 18-bit after multiply

3.2.2 The Update Coefficient

The key coefficient in all three update equations is $c^2 \times \Delta t / \Delta x$ (for the Ex and Ey updates) and $\Delta t / \Delta x$ (for the Bz update). With $c_eff = 0.5$ and $\Delta t = 0.35$, $\Delta x = 1$:

$$Cb_E = c_eff^2 \times \Delta t / \Delta x = 0.25 \times 0.35 = 0.0875$$
$$Cb_B = \Delta t / \Delta x = 0.35$$

These coefficients are stored as fixed-point constants in the Verilog — no division occurs during the simulation loop.

3.2.3 Overflow Protection

Field values near sources can briefly exceed the Q3.13 range. Add saturation logic: if the result of an addition exceeds the maximum representable value, clamp to the maximum. In Verilog this is a simple comparison after each accumulation:

```
// Saturation example (18-bit Q3.13)
wire signed [17:0] result;
wire signed [17:0] saturated;
assign saturated = (result > 18'sh1FFFF) ? 18'sh1FFFF :
                  (result < -18'sh20000) ? -18'sh20000 : result;
```

3.3 The FDTD Engine — Verilog Module Structure

3.3.1 Top-Level Module Hierarchy

Module	Function	Inputs	Outputs
fdtd_top	Top-level wrapper, connects all blocks	Clock, reset, AXI registers	Pixel stream to HDMI
fdtd_engine	Core FDTD update sequencer	Field BRAMs, coefficients	Updated field BRAMs
bz_updater	Computes Bz update at each cell	Ex, Ey at 4 neighbours	New Bz value
ex_updater	Computes Ex update at each cell	Bz at 2 neighbours	New Ex value
ey_updater	Computes Ey update at each cell	Bz at 2 neighbours	New Ey value

Module	Function	Inputs	Outputs
source_injector	Injects source waveforms	Time step counter, source params	Field delta to add
pml_multiplier	Applies PML damping at boundaries	Cell position, field value	Attenuated field value
renderer_3d	3D surface rendering pipeline	Field BRAM, rotation angles	AXI stream pixel output
poynting_calc	Computes $S = E \times B$ per pixel	Ex, Ey, Bz arrays	Sx, Sy, Sz arrays

3.3.2 The FDTD Engine Sequencer

The engine steps through the grid systematically. The sequencer is a state machine with the following states:

1. IDLE — waiting for start signal from Python (new boundary conditions available)
2. UPDATE_BZ — iterate through all (i,j), compute and write new Bz. Takes $NX \times NY = 65,536$ clock cycles.
3. INJECT_SOURCE — after Bz update, add source contributions at source positions. Takes N_{sources} clock cycles.
4. APPLY_PML_BZ — multiply Bz at boundary cells by decay coefficient. Takes $\sim 4 \times 8 \times NX$ clock cycles.
5. UPDATE_EX — iterate through all (i,j), compute and write new Ex. Takes $NX \times NY$ clock cycles.
6. UPDATE_EY — iterate through all (i,j), compute and write new Ey. Takes $NX \times NY$ clock cycles.
7. APPLY_PML_E — multiply Ex, Ey at boundary cells by decay coefficient.
8. STEP_DONE — increment time step counter, signal renderer to start, go to IDLE or loop.

Total clock cycles per time step: approximately $4 \times 65,536 + \text{overhead} \approx 270,000$ clock cycles. At 100MHz: 2.7ms per time step. At 60fps (16.7ms per frame): approximately 6 full FDTD time steps per frame. This gives visibly smooth wave animation.

3.3.3 The Bz Updater — Verilog

The core arithmetic of the FDTD update. This module is called once per cell per time step:

```

module bz_updater (
    input  clk,
    input  signed [17:0] bz_current,    // Bz at (i,j)
    input  signed [17:0] ex_j,          // Ex at (i,j)    - note Yee offset
    input  signed [17:0] ex_j1,         // Ex at (i,j+1)
    input  signed [17:0] ey_i,          // Ey at (i,j)
    input  signed [17:0] ey_i1,         // Ey at (i+1,j)
    input  signed [17:0] Cb,            // =  $\Delta t / \Delta x$  in Q3.13
    output reg signed [17:0] bz_new
);
    wire signed [35:0] curl_E;

```

```

// curl_E = (Ex(j+1) - Ex(j))/Δy - (Ey(i+1) - Ey(i))/Δx
// With Δx=Δy=1: curl_E = ex_j1 - ex_j - ey_i1 + ey_i
assign curl_E = ex_j1 - ex_j - ey_i1 + ey_i;
// Bz update: Bz_new = Bz - Cb × curl_E
always @(posedge clk)
    bz_new <= bz_current - ((Cb * curl_E[17:0]) >>> 13);
endmodule

```

3.3.4 The Ex Updater — Verilog

```

module ex_updater (
    input  clk,
    input  signed [17:0] ex_current,
    input  signed [17:0] bz_j,           // Bz at (i,j)
    input  signed [17:0] bz_j_minus1,   // Bz at (i,j-1)
    input  signed [17:0] Cb_E,          // = c2 × Δt/Δx in Q3.13
    output reg signed [17:0] ex_new
);
    wire signed [17:0] curl_B;
    assign curl_B = bz_j - bz_j_minus1; // ∂Bz/∂y × Δy
    always @(posedge clk)
        ex_new <= ex_current + ((Cb_E * curl_B) >>> 13);
endmodule

```

3.3.5 Source Injection Logic

```

module source_injector #(parameter N_SOURCES = 4) (
    input  clk,
    input  [31:0] time_step,
    input  [7:0]  src_x [0:N_SOURCES-1],
    input  [7:0]  src_y [0:N_SOURCES-1],
    input  signed [17:0] src_amp [0:N_SOURCES-1],
    input  signed [17:0] src_phase [0:N_SOURCES-1],
    input  signed [17:0] f0,          // normalised frequency in Q3.13
    output reg signed [17:0] delta_ex, // amount to add to Ex at source
    output reg [7:0] src_xi, src_yi  // which cell to inject
);
    // Compute sin(2π f0 n Δt + phase) using CORDIC IP
    // Xilinx CORDIC configured for sin/cos output
    // One output per clock cycle — iterate through sources
endmodule

```

3.4 The Boundary Conditions — Physical Measurement

This is where the physical conducting paper measurement connects to the FDTD simulation.

3.4.1 What the Measurement Provides

The conducting paper with a hand probe gives you (x, y, V_measured) — the potential at any point the user touches. The Python layer on the Pynq CPU receives this via the resistive touchscreen's ADC outputs and the probe ADC reading.

3.4.2 How to Use Measurement as Boundary Condition

The measured potential defines the E field at the electrode positions. In the FDTD simulation, these become Dirichlet boundary conditions — grid cells where the field is forced to a fixed value rather than being updated by the FDTD equations.

For each electrode at position (xe, ye) with voltage Ve:

- The Ex and Ey values at (xe, ye) are held fixed at values consistent with Ve
- The FDTD update equation is NOT applied at these cells — they are skipped
- The surrounding cells ARE updated normally — they respond to the electrode's fixed field

In Verilog, this is implemented by a boundary condition mask — a 256×256 bit array stored in block RAM where a '1' indicates a fixed cell and a '0' indicates a free cell updated by FDTD. The update sequencer checks this mask before applying the update equation.

3.4.3 From Voltage to Field Value

The measured voltage V maps to Ex and Ey at the electrode position. Since $E = -\nabla V$, a positive voltage region surrounded by lower voltage has E pointing outward. For the FDTD initialisation:

$$E_x \text{ at electrode} \approx -(V_{\text{right}} - V_{\text{left}}) / (2\Delta x)$$

$$E_y \text{ at electrode} \approx -(V_{\text{above}} - V_{\text{below}}) / (2\Delta y)$$

Where the surrounding values come from the Laplace solution (the conducting paper measurement). This correctly initialises the E field consistent with the measured potential.

3.4.4 The Comparison Mode

In comparison mode, the FPGA simultaneously shows:

- Left panel: static Laplace solution from physical measurement (Jacobi solver or direct measurement)
- Right panel: FDTD dynamic solution evolving from the same initial conditions

Initially they are identical. As the FDTD evolves, it shows how the field responds dynamically to changes — wave propagation, radiation, ringing — that the static Laplace solution cannot show. This directly motivates why Maxwell's equations are needed beyond Laplace.

3.5 Poynting Vector in FDTD

In FDTD, the Poynting vector is computed from the time-averaged fields. Since E and B are offset by half a time step in the leapfrog scheme, you must interpolate to get them at the same instant before taking the cross product.

3.5.1 Time Averaging

$$E_{\text{avg}}^n = (E^{(n-\frac{1}{2})} + E^{(n+\frac{1}{2})}) / 2 \approx E^n \quad (\text{to first order})$$

For the TE mode with only Bz as the magnetic component:

$$S_x = E_y \times B_z / \mu_0$$

$$S_y = -E_x \times B_z / \mu_0$$

$$S_z = 0 \quad (\text{no out-of-plane Poynting flux in 2D TE mode})$$

The magnitude of power flow in the plane is:

$$|S| = \sqrt{S_x^2 + S_y^2} / \mu_0 = |E \times B| / \mu_0$$

This is computed per pixel by the Poynting calculator module and stored in a separate block RAM for the renderer.

3.5.2 Time-Averaged Poynting Vector

For a sinusoidal steady state (after the simulation has run for many cycles), the time-averaged Poynting vector is more physically meaningful than the instantaneous value:

$$\langle S \rangle = (1/2) \operatorname{Re}(E \times B^*) / \mu_0$$

In FDTD this is approximated by accumulating the instantaneous S_x and S_y over one full source period and dividing. The FPGA maintains running averages using a sliding window accumulator — subtract the oldest value, add the newest, divide by window length. This gives a smooth, stable Poynting vector display that shows the net energy flow direction rather than the oscillating instantaneous values.

4. 3D Visualisation of FDTD Fields

4.1 What Each Field Looks Like as a 3D Surface

4.1.1 $E_x(x,y)$ and $E_y(x,y)$ — Electric Field Components

E_x and E_y oscillate between positive and negative values as the wave propagates. As 3D surfaces they look like rippling sheets — peaks and valleys sweeping across the grid as the wave moves. Near a point source, the surface shows circular ripples propagating outward, diminishing in amplitude with distance. For two sources, the surface shows an interference pattern — alternating ridges and valleys where the waves constructively and destructively interfere.

4.1.2 $B_z(x,y)$ — Magnetic Field

In TE mode, B_z is perpendicular to the simulation plane. Its 3D surface also shows the wave propagation pattern but phase-shifted by 90° relative to E in the direction of propagation. Near the source, B_z is strongest where E_x and E_y are weakest — the energy oscillates between E and B storage as the wave propagates. Far from the source, E and B have equal amplitude — this is the far-field condition where the wave is a plane wave.

4.1.3 $|S|(x,y)$ — Poynting Vector Magnitude

The Poynting surface shows where power flows most intensely. Near the source it shows a complex near-field pattern with energy oscillating back and forth. In the far field it shows smooth, outward-flowing power. Between two interfering sources, the Poynting surface shows bright fringes where constructive interference directs energy and dark fringes where destructive interference blocks it. This is the most physically informative surface — it shows where the electromagnetic energy actually goes.

4.1.4 The Time Dimension

Unlike the static Laplace surfaces, the FDTD surfaces are animated — they change every frame as the wave evolves. The user sees the wave propagate outward from the source in real time. Moving an electrode (source) shows a new wave front emanating from the new position. Changing the drive frequency visibly changes the wavelength on screen. This animation is the most powerful demonstration of the FPGA's advantage over the static Laplace approach.

4.2 Rendering the Animated 3D Surface

4.2.1 The Challenge Versus Static Rendering

For the static Laplace case, the field changes slowly (only when electrodes move). For FDTD, the field changes every frame — the entire surface reshapes 60 times per second. The renderer must read the complete field BRAM, build the mesh, project all vertices, and rasterise all triangles within 16.7ms. This is tight but achievable for a 256×256 grid with careful pipelining.

4.2.2 Reduced Resolution Rendering

Rather than rendering all $256 \times 256 = 65,536$ points as a mesh (which would create 130,000 triangles — too many for real-time rendering), subsample the field to a 64×64 rendering grid. Every 4th point in x and y is used, giving 4096 points and 8000 triangles. Intermediate values are bilinearly interpolated for colour mapping, giving smooth colour even on the coarser render mesh.

4.2.3 False Colour for Oscillating Fields

Unlike the static Laplace surface where values are always positive, FDTD fields oscillate between positive and negative. The colour map must handle this:

- Positive E_x : warm colours (yellow $\rightarrow$ red) — field pointing in $+x$ direction
- Negative E_x : cool colours (cyan $\rightarrow$ blue) — field pointing in $-x$ direction
- Zero: black or grey — field node
- For $|S|$: black $\rightarrow$ purple $\rightarrow$ orange $\rightarrow$ white — power always positive

This bipolar colour map is used in all professional EM simulation software. It immediately shows wave crests (red) and troughs (blue) propagating across the grid.

4.2.4 Phase Animation

An alternative to height mapping for oscillating fields is to show the field purely through colour on a flat grid — like a heat map that ripples. This is simpler to render (no 3D projection needed) and can show the full 256×256 resolution without subsampling. Use this as a fallback mode if the full 3D rendering pipeline proves too slow.

Show both: a small 3D perspective view in one corner showing the surface shape, and a large flat colour map showing full-resolution field animation. The combination gives both structural information (3D view) and detailed wave pattern (2D map).

4.3 Displaying Multiple Fields Simultaneously

With FDTD you have more interesting things to show than with Laplace. The display is divided into regions:

Display Region	Content	Update Rate
Top left (quarter screen)	E_x field — 3D rotating surface	60fps animated
Top right (quarter screen)	B_z field — 3D rotating surface	60fps animated
Bottom left (quarter screen)	$ S $ Poynting — 3D rotating surface	60fps animated
Bottom right (quarter screen)	2D flat field map — full resolution, false colour	60fps animated
Overlay (all regions)	Source positions marked, field values at cursor, frequency readout	60fps

All four regions rotate and animate simultaneously. The user controls the 3D rotation with potentiometers — all three 3D surfaces rotate in sync (same θ and ϕ applied to all three). This

helps the viewer understand the relationship between the three fields — they are looking at the same spatial domain from the same angle, showing different field components.

5. Physical Measurement Integration

5.1 Conducting Paper Setup

Teledeltos paper (graphite-coated conducting paper) is the resistive medium. It is uniform, stable, and requires no water. Two electrodes clipped to opposite edges (or any positions on the paper) create the boundary conditions. The user probes the surface with a thin metal tip connected to the ADC.

5.2 XY Position Sensing — Resistive Touch Panel

A resistive touch panel placed over the conducting paper simultaneously senses probe XY position. When the probe tip presses through the panel onto the paper, the touch panel reports (x_norm, y_norm) as two voltages on its Y+ and X+ pins, and the probe ADC reads the voltage V at that point.

Signal	Source	Pynq Interface	What It Gives
x_norm	Touch panel X+ pin	XADC channel 1	Probe x position (0.0 to 1.0)
y_norm	Touch panel Y+ pin	XADC channel 2	Probe y position (0.0 to 1.0)
V_measured	Probe tip ADC	XADC channel 0	Potential at probe position
Electrode A voltage	Drive circuit output	Fixed — set by potentiometer	Boundary condition at electrode A
Electrode B voltage	Drive circuit output	Fixed — set by potentiometer	Boundary condition at electrode B

5.3 How the Measurement Feeds FDTD

The Python control loop runs at 50Hz and:

9. Reads the probe position (x, y) and voltage V from the three XADC channels.
10. Maps (x, y) from normalised 0–1 range to grid indices (0–255, 0–255).
11. Writes V to a boundary condition register: position (ix, iy) → voltage V.
12. Writes electrode A and B positions and voltages to boundary condition registers.
13. FPGA reads these registers and holds the corresponding grid cells at fixed values during FDTD update.

The result: wherever the user touches the paper, a new boundary condition is injected into the FDTD simulation at that grid point. Moving the probe across the paper in real time draws boundary conditions into the simulation — like drawing with an electric potential.

5.4 Three Operating Modes

Mode	Physical Input	FPGA Computation	What User Sees
Static Laplace	Probe scans paper surface	Jacobi solver converges to static solution	Static 3D potential surface — matches Laplace exactly
FDTD Static	Same as above	FDTD with same boundary conditions (no AC drive)	Should match Laplace — validates FDTD implementation
FDTD Dynamic	Electrode driven at frequency f_0	Full FDTD wave propagation	Animated waves propagating from electrode positions
FDTD + Probe	User draws with probe	Each probe touch injects boundary condition	Waves respond in real time to probe position — interactive wave tank
Comparison	Probe scans paper	Laplace and FDTD run simultaneously	Side-by-side comparison: static vs dynamic EM field

5.5 The Interactive Wave Tank

Mode 4 above is the most impressive demo. The FDTD simulation runs continuously with a source at the centre. The user touches the conducting paper with the probe — this injects a boundary condition (a wall or obstacle) into the simulation at that point. Waves diffract around the probe tip in real time. Moving the probe slowly creates a moving obstacle that the waves respond to continuously.

This is physically analogous to touching the surface of a ripple tank — the wave pattern reshapes around the probe tip as it moves. The difference is that these are electromagnetic waves computed by Maxwell's equations on your FPGA, not water waves. The physics is identical in mathematical form.

PART 6 — IMPLEMENTATION PLAN

6. Implementation Plan

6.1 Revised Work Split for Maxwell/FDTD

Student	Role	Core Deliverable
EEE 1	Signal generator + electrode drive	AC drive circuit, electrodes on conducting paper, patch panel
EEE 2	Probe system + ADC interface	Resistive touch panel, probe ADC, XADC wiring, Python readout
IEE 1	FDTD engine — Bz and Ex/Ey updaters	Core FDTD Verilog: bz_updater, ex_updater, ey_updater, sequencer
IEE 2	Source injection + PML + Poynting	source_injector, pml_multiplier, poynting_calc modules
IEE 3	3D renderer — projection + rasterisation	renderer_3d: mesh build, rotation, perspective, Phong rasterise
IEE 4	Python control + Vivado integration	AXI layer, boundary condition writes, mode switching, system integration

6.2 Build Order — 6 Weeks

Week	IEE Milestone	EEE Milestone
1	Set up Vivado. Build starter overlay. HDMI test pattern working. Define AXI register map and BRAM layout — this is the interface all blocks depend on.	Order components. Build AD9833 SPI interface. Verify sine wave on oscilloscope. Acquire Teledeltos paper and resistive touch panel.
2	IEE 1: Implement FDTD engine for a single field component (Bz only, simple 1D case first). Verify update equation numerically. IEE 4: Implement BRAM access from Python CPU.	EEE 1: Build 2-channel electrode drive on breadboard. Test on conducting paper with multimeter. EEE 2: Wire resistive touch panel to XADC. Read X/Y coordinates in Python.
3	IEE 1: Full 2D FDTD engine — all three components, full grid. Simulate in Icarus Verilog first. IEE 2: Implement source injector — verify wave propagation in simulation.	EEE 2: Build probe ADC circuit. Wire to XADC. Verify (x, y, V) readout in Python. Run first static field measurement on paper.

Week	IEE Milestone	EEE Milestone
4	IEE 2: Implement PML absorbing boundaries. IEE 3: Implement flat 2D colour map renderer first (simpler than 3D). Verify animated wave on HDMI.	EEE 1+2: Integration — connect drive circuit to paper, probe to XADC. Verify boundary conditions injected into FDTD from probe touch.
5	IEE 3: Implement full 3D rendering pipeline (projection, rotation, Phong). IEE 2: Implement Poynting vector calculator. All four display panels working.	Integration testing. Verify wave pattern changes when electrodes move. Demo sequence rehearsal.
6	Polish: comparison mode, field value readout overlay, multiple colour schemes. Performance optimisation. Full demo rehearsal.	Polish physical apparatus. Make patch panel clean and labelled. Prepare demo script. Test robustness of probe connection.

6.3 Simulation-First Development Strategy

Every FDTD Verilog module must be verified in software simulation before loading to hardware. FPGA build times are 15–20 minutes — debugging on hardware is impractical. Use this workflow:

14. Write the Verilog module.
15. Write a testbench that exercises the module with known inputs.
16. Simulate with Icarus Verilog — verify outputs match hand-calculated values.
17. Run simulation for 1000+ time steps — verify wave propagates, doesn't blow up.
18. Check that energy is conserved (field amplitude doesn't grow without a source).
19. Only then generate the bitstream and load to hardware.

For the FDTD engine specifically: simulate a 1D version first (just a row of cells, waves propagating left and right). This is much easier to debug than the full 2D case and verifies the core arithmetic is correct.

6.4 Key Debug Checks

Check	How to Test	What Failure Looks Like
CFL condition satisfied	Compute $\Delta t / (\Delta x / c_{\text{eff}} \times \sqrt{2})$ — must be < 1	Field values blow up exponentially within a few hundred steps
Fixed-point overflow	Monitor max field value in simulation	Display fills with maximum value colour (saturated)
Wave speed correct	Measure how many time steps a wave takes to cross the grid	Wave speed $\neq c_{\text{eff}} \times \Delta x / \Delta t$
PML working	Run simulation without source for 1000 steps after source off — field should decay to zero	Reflections from boundaries visible as standing wave pattern

Check	How to Test	What Failure Looks Like
Poynting direction correct	For a wave propagating in +x direction, Sy should be zero, Sx should be positive	Poynting arrows point in wrong direction
BRAM addressing	Read back values written by CPU — verify Python reads match what was written	Boundary conditions appear at wrong grid positions

6.5 FPGA Resource Estimate

Resource	Available (Pynq-Z1)	Estimated Usage	Headroom
Block RAM (36Kb)	140	~90 (field arrays + pixel buffer)	50 BRAMs spare
DSP48E1 slices	220	~60 (multipliers in updaters + renderer)	160 spare
LUT	53,200	~25,000 (control logic + routing)	28,000 spare
Flip-Flop	106,400	~20,000	86,000 spare
Clock frequency	Up to 200MHz	100MHz target	2× headroom

Resource Verdict

The Pynq-Z1 has sufficient resources for the full FDTD + 3D renderer. The block RAM for three 256×256×16-bit field arrays (~90 BRAMs) is the tightest constraint, using 64% of available BRAM. If needed, reduce grid to 192×192 to give more margin — wave patterns remain clearly visible.

7. Demo Scenarios — What the Examiner Sees

7.1 Scenario 1 — Static Field Validation

Start with FDTD in static mode (no AC drive). Place two electrodes on the conducting paper. The FDTD simulation converges to the Laplace solution. The 3D surface on screen shows the classic dipole potential landscape — one peak, one valley, smooth saddle between. Switch to comparison mode: left panel shows the Laplace solver output, right panel shows the FDTD static solution. They match. This validates the FDTD implementation and demonstrates that Maxwell's equations reduce to Laplace in the static limit.

7.2 Scenario 2 — Wave Propagation

Switch to dynamic mode. The electrode is now driven at frequency f_0 . Immediately, circular wavefronts propagate outward from the source position across all three 3D surfaces. The E_x surface shows the oscillating electric field rippling outward. The B_z surface shows the magnetic field, 90° out of phase. The Poynting surface shows smooth outward energy flow — the radiation pattern of a point source. The examiner can rotate all three surfaces simultaneously to inspect the wave from any angle.

7.3 Scenario 3 — Two-Source Interference

A second source is activated (second electrode, driven at same frequency). Immediately the interference pattern appears — the classic alternating fringes of constructive and destructive interference. The Poynting surface is particularly striking: bright fringes where energy flows strongly outward between the interference maxima, and dark nodes where destructive interference prevents energy flow. This is the EM equivalent of Young's double-slit experiment, computed in real time on the FPGA.

7.4 Scenario 4 — Interactive Wave Tank

The user picks up the conducting paper probe and touches it to the surface. A boundary condition (a reflecting obstacle) is injected at that point. The wave pattern immediately diffracts around the probe tip — visible as a shadow region behind the tip and curved wavefronts around its sides. Moving the probe slowly shows the diffraction pattern tracking the obstacle in real time. This is the most interactive and visually dramatic part of the demo.

7.5 Scenario 5 — Poynting Vector Deep Dive

With two sources creating an interference pattern, switch to Poynting-only full-screen mode. The time-averaged Poynting vector magnitude fills the screen as a 3D surface. Peaks of power flow appear between the interference maxima — channels where electromagnetic energy is directed by the interference pattern. The user rotates the surface to see the power flow landscape from all angles. This directly demonstrates the project's title claim: we can visualise where electromagnetic energy goes, not just where the fields are strong.

The Central Message of the Demo

Laplace's equation tells you where charges sit in equilibrium. Maxwell's equations tell you how electromagnetic energy moves through space and time. The FPGA solves Maxwell's equations — 65,536 coupled differential equations simultaneously — fast enough to show the solution animating in real time at 60 frames per second. A CPU cannot do this. That is why the FPGA is here.

8. Component List — Within £60 Budget

Item	Qty	Cost	Notes
Teledeltos conducting paper (A2 roll)	1	£12	Core sensing medium — resistive, uniform
Resistive touch panel (7-inch)	1	£8	XY probe position sensing
AD9833 DDS module	1	£4	1kHz sine wave generator
MCP6004 quad op-amp	4	£4	Probe buffer + filter (cheaper than 64 for water approach)
TL071 op-amp	6	£3	Drive circuit buffers and amplifier
10kΩ potentiometers	8	£6	Electrode amplitude, rotation control, wire position
SPDT toggle switches	4	£3	Electrode polarity selection
Resistors assorted (0805 SMD)	1 pack	£3	Filters, dividers, gain setting
Capacitors 10nF (0805 SMD)	10	£2	Bandpass filter
Crocodile clip leads	6	£3	Electrode connections to paper
Banana sockets	4	£2	Drive circuit outputs
Breadboard (full size)	1	£4	Drive circuit prototyping
Stripboard	1	£2	Probe ADC circuit
Jumper wires	1 pack	£2	Breadboard connections
Small project box	1	£3	Enclose drive circuit cleanly for demo
Pynq-Z1	1	Borrowed	From EEE stores
HDMI cable	1	In kit	From Pynq kit
Total		~£61	Slightly over — use department resistors/caps to save £3

Budget Note

The Maxwell/FDTD approach is significantly cheaper than the water tray approach (no sense grid PCB, no 64-channel mux chain, no perspex tray). The conducting paper + resistive touch panel replaces ~£30 of hardware with ~£20 of hardware. The saved budget can go toward a nicer project box for the control panel.